C4A 2024 2510 Midterm Review

Created by Harrison & Tarasios

With thanks to Chris Taehyung and Jeeho

BCIT Coding For All

In This Review

- Material Review (~15 min)
- Tips for Studying and Performing Well (~5 min)
- Practice Midterm Problems (~60 min)

Topics Covered

- Data types
- Pointers
- Memory
- Arrays
- Structs
- ~~Linked Lists~~

BCIT Coding For All

Data Types

Character Types:

- `char`: (1B) int type that can represent both basic characters (like 'A' and 'z') and special characters (like '\n').

Integer Types:

- `int`: (2-4B) Integer, machine dependent, but often 32 bits.
- `short`: (2B)Short integer, guaranteed to be at least 16 bits.
- `long`: (4+B) Long integer, guaranteed to be at least 32 bits.
- `long long`: (8+B)Extended long integer, guaranteed to be at least 64 bits.
- Each of the integer types can be modified with `signed` or `unsigned` to represent negative or non-negative numbers, respectively.

Floating-Point Types:

- `float`: Single-precision floating-point number.
- `double`: Double-precision floating-point number.
- `long double`: Extended precision floating-point number.

Boolean Type:

- `_Bool` or `bool`: Represents true or false values. Requires `#include <stdbool.h>` for the `bool` type.
- Can represent as a 1 or 0 for True/False instead

Void Type:

- `void`: Represents the absence of type

BCIT Coding For All

Data Types: Pointers

Basic Pointer Types

- A pointer type is declared by placing an asterisk (*) before the pointer name. The type specifies the data type of the variable the pointer will point to.
- General Syntax: `type *pointerName;`

E.g.

```
char* ptr
```

Array Pointers

- When you declare an array, C automatically sets up a pointer to the first element of the array

E.g.

```
int arr[5] = {1, 2, 3, 4, 5};
int *array_ptr = arr;                // Pointer to the first element of arr
int first_value = *array_ptr;        // Dereferencing gives the first element (1)
int second_value = *(array_ptr + 1); // Points to the second element (2)
int third_value = array_ptr[2];      // Subscripting the pointer for the third element (3)
```

NOTE: [ ] always dereferences automatically, can be used on any pointer

Null Pointer

- Explanation: A null pointer does not point to any object or function and is often used for error handling and initialization.
- Syntax: `type *pointerName = NULL;`

E.g.

```
int *ptr = NULL;
```

BCIT Coding For All

Data Types: Pointers

Pointers to pointers (double pointers)

- A double pointer can store the address of another pointer, allowing for multi-level indirection.
- Syntax: `type **pointerName;`

E.g.

```
float** ptr
```

You may not have covered the following, don't worry about it for the midterm, just know there are other types of pointers

Pointers and `const`

- Pointer to a Constant: The data pointed to cannot be changed.
  - Syntax: `const type *pointerName;`
- Constant Pointer: The pointer itself cannot change to point to another address.
  - Syntax: `type *const pointerName;`
- Constant Pointer to a Constant: Neither the pointer address nor the data pointed to can be changed.
  - Syntax: `const type *const pointerName;`

Function Pointers: Syntax: `returnType (*pointerName)(parameterTypes);`

E.g. `void (*funcPtr)(int, float);`

Void Pointers: Syntax: `void *pointerName;`

- A void pointer can hold the address of any data type and is mainly used for generic data operations.

E.g. `void *ptr;`

BCIT Coding For All

Pointer Review

Pointer: variable that stores the address of another variable in memory

Key understanding:

- a pointer variable is NOT the same as a variable.
- Pointer variables have pointer types. To the compiler both pointers and pointer types are seen and treated differently.
- Pointers are “lower level”, they access memory locations directly similarly to assembly language (require fewer assembly instructions).
- Regular variables are more abstracted and are associated with a primitive type.
- Are essential for dynamic memory allocation

Pointer Review: Operators

Address-of: **&**

- "Give me your ***address***"

```
char *ptr;
ptr = &var;
```

Dereference Operator: *****

- "Give me your ***value***"

```
int *ptr;  // * here is not the D.O.
*ptr = 5;
int value = *ptr;
```

Array Subscript Operator: **[ ]**

- "Give me the ***value*** of element at this position"
- Think of it as automatically having * with it

```
int arr[5] = {10, 20, 30, 40, 50};
int x = arr[2];

int *ptr = arr;
int y = ptr[2];
```

Equality and Relational Operators: **==, !=, <, >, <=, >=**

- Compare two pointers' memory addresses.

```
if (ptr1 == ptr2) { ... }
```

Assignment Operator: **=**

- Assigns the value of one pointer to another.

```
ptr2 = ptr1;
```

BCIT Coding For All

Pointer Review (3)

Declaration of an int pointer
int *ptr;

Initialization of a pointer
int var = 5;
ptr = &var; //points to “var”

Printing the pointer value and stored address

printf("%p", ptr); //0x7ffd71bf0fa4 address of “var”

printf("%d", *ptr); //5 value of var

SIDENOTE: good practice to print an address in a pointer by casting to a void pointer

printf("%p", (void *)ptr); //0x7ffd71bf0fa4

BCIT Coding For All

Pop Quiz

Can I do the following?

```
int *ptr = 5;
```

What is going on here?

Pop Quiz

Can I do the following? NO

```
int *ptr = 5;
```

(*) here is not operating on ptr, it is the data type int*
Pointer data types are unique and necessary for pointers.

C4A
BCIT Coding For All

Pop Quiz

How about this?

```
int *ptr;
*ptr = 5;
```

BCIT Coding For All

Pop Quiz

How about this? NOPE DON’T DO THIS!

```
int *ptr;
*ptr = 5;
```

Will likely seg fault… but it might not. You better hope it does!

This is undefined behaviour and may have different outcomes on different machines. For security reasons, an uninitialized pointer has a random bit pattern that could possibly point to a random spot in memory, this is likely illegal (seg fault). But it could write this value to a location in memory the program has access to. In some contexts, this could corrupt your device.

Pop Quiz

How about this?

```
int var;
int *ptr = &var;
*ptr = 5;
```

C4A
BCIT Coding For All

Pop Quiz

How about this? YES

```
int var;
int *ptr = &var;
*ptr = 5;
```

Indirect assignment of 5 to var through the pointer

General advice: initiate pointers when you instantiate them

BCIT Coding For All

Pointer Review: Arithmetic

- Increment (`ptr++` or `ptr += n`): Moves the pointer to the next memory location, or `n` locations forward, based on the size of the type it points to.
- Decrement (`ptr--` or `ptr -= n`): Moves the pointer to the previous memory location, or `n` locations backward.
- Addition (`ptr + n`): Calculates a new address `n` elements ahead of the current pointer position, returning a new pointer value. Does not alter the original pointer.
- Subtraction (`ptr - n`): Generates a new address `n` elements behind the current pointer position, yielding a new pointer value. The original pointer remains unchanged.
- Difference Between Pointers (`ptr2 - ptr1`): Computes the number of elements between two pointers `ptr2` and `ptr1` of the same type. Useful for finding the length of a segment within an array.

E.g. of addition:

```
int array[] = {10, 20, 30, 40, 50, 60, 70};
int *ptr = array; // Equivalent to &array[0]
int *ptr_plus = ptr +3; // ptr_plus points to array[3]
```

BCIT Coding For All

Memory Review

Stack

- Automatic: Variables declared within a function are allocated on the stack.
- Scope-Limited: Memory is automatically freed when the variable goes out of scope.
- Fast & Efficient: Ideal for temporary data with known size.

Heap

- Dynamic: Memory is allocated and freed manually using `malloc`, `calloc`, `realloc`, and `free`
- Program-Controlled Lifetime: Memory stays allocated until explicitly freed, regardless of scope.
- Flexible: Suitable for variable-sized data or data that needs to persist beyond its initial scope.

Dynamic Memory Allocation

malloc: allocates a specified number of contiguous uninitialized bytes

```
int* num = malloc(sizeof(int));
```

calloc: allocates a specified number of contiguous bytes initialized to 0 (useful for arrays)

```
char* num = calloc(10, sizeof(char));
```

realloc: resizes previously allocate memory

```
int *array = malloc(5 * sizeof(int));
array = realloc(array, 10 * sizeof(int));
```

free: frees previously allocated memory to the heap

```
int *data = malloc(100 * sizeof(int));
// Use the allocated memory...
free(data);
```

BCIT Coding For All

Dynamic Memory Allocation

Why is this necessary?

- Allocates memory at runtime, instead of compile-time
  - Compiler does not know how much memory is needed
  - Stack and static/global memory are fixed at program start. The size of the stack is predetermined, and large allocations can cause stack overflow.
- Stack memory is strictly managed using the **LIFO** principle. Once a function exits, the memory used for its local variables is automatically reclaimed, and you can't control when or how this memory is freed.
- Global and static variables are allocated for the entire lifetime of the program, even if they are only needed for a short period. This can lead to memory waste in long-running programs.

Arrays in C

Declaration(best to initialize here as well)

```
int numbers[5];
char name[20];
float *dec = malloc(n * sizeof(float));  //an allocation that happens at runtime, will need to free later
```

Initialization

```
int nums[] = {1, 2, 3, 4, 5};        // Array of 5 integers
char course[14] = "C Programming";   // String (char array) automatically has +1byte char: '\0'
if (dec != NULL) {                   // Array of n floats
        for(int i = 0; i < n; i++) {
            dec[i] = i * 1.0;
        }
strcpy(name, "C Programming");          //copy content into array given name is large enough (<string.h>)
memcpy(numbers, nums, sizeof(nums)); // Correct usage of memcpy to copy contents (<string.h>)
```

Accessing Elements

```
numbers[0] = 10;
int five = numbers[4];
int* five_ptr = &numbers[4];
int *ptr = name + 5;     //'g'
float x = dec[n-1]
```

BCIT Coding For All

Structs

C's version of an class (does not include methods, access control, inheritance, polymorphism or refunds)!
Groups data together. Is a contiguous block of data.

```
struct StructName {
    dataType member1;
    dataType member2;
    ...
}
```

E.g.

```
struct Person {
    char name[50];
    int age;
    float salary;
};

int main() {
  struct Person s1 = {"John Doe", 30, 85.5};
  struct Person s2;
  s2 = s1;            //performs a shallow copy, makes a second struct with same data
                      // pointers in a copied struct will point to the same addresses
```

An alternative naming convention:

```
typedef struct Person {
    char name[50];
    int age;
    float salary;
} Person;

int main() {
   Person person1; //
}
```

BCIT Coding For All

Structs: Arrow Operator

Dot operator (.) operator: access a member of a structure or a union directly

Arrow operator (->) Dereferences a pointer to a structure and then accesses a member of that structure.

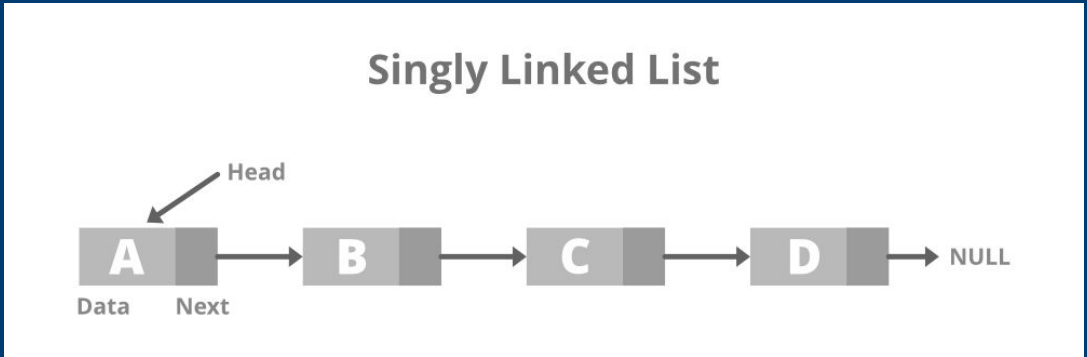
```
struct Point {
    int x;
    int y;
};

int main() {
    struct Point p;
    p.x = 10;
    p.y = 20;
    printf("The point is at (%d, %d)\n", p.x, p.y);

    struct Point *ptr = malloc(sizeof(struct Point));
    ptr->x = 11;
    (*ptr).y = 21

    return 0;
}
```

BCIT Coding For All

Linked-Lists

Very fundamental data structure, if there's one thing to practice for your midterm, it's linked-lists

- Many other data structures are based off of linked lists
- With practice, it's not too hard to master, but requires good understanding of pointers
- Do as many problems on leetcode as you can: https://leetcode.com/tag/linked-list/
- Historically there is one or more major linked-list questions worth roughly half the marks on the midterm
- Expect it to be leetcode medium level, but graded somewhat generously based on the correctness of your logic



Things you *might* need for your linked list:

Node
Data
Next pointer
Prev pointer (for doubly-linked lists)
Head
Tail (optional, for efficiency)
Create function
Insert function
Delete function
Search function
Free function (for memory cleanup)
Length function (optional, for convenience)

BCIT Coding For All

Why Linked-Lists

Dynamic & Flexible
- Grows as needed, very mutable
- Ideal for unknown data sizes

Efficient Operations
- Quick insertions/deletions O(1)-O(n)
- No element shifting required

Types & Uses
- Singly: Simple, unidirectional
- Doubly: Easy reversals, bidirectional
- Circular: Continuous access

Key Advantages
- Avoids memory fragmentation
- Supports dynamic data structures like stacks and queues

Considerations
- Extra memory for pointers
- Sequential access, no direct indexing

Bottom Line:
Linked lists excel in scenarios requiring dynamic memory and frequent modifications.

Singly Linked-List

```
//the individual components of the linked list
struct Node {
   int data;
   struct Node* next;
};

//function to create a node with given data
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = data; // Assign data to the new node
    newNode->next = NULL; // Initialize next as NULL
    return newNode;
}
```

```
int main() {
    struct Node* head = createNode(1);
    struct Node* second = createNode(2);
    struct Node* third = createNode(3);

    // Linking nodes
    head->next = second;
    second->next = third;

    // LL is: 1 -> 2 -> 3 -> NULL

    return 0;
}
```

BCIT Coding For All

Doubly Linked-List

```
typedef struct Node {
    int data;
    struct Node* prev; // Pointer to the previous node
    struct Node* next; // Pointer to the next node
} Node;

Node* createNode(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->prev = NULL; // Initialize prev as NULL
    newNode->next = NULL; // Initialize next as NULL
    return newNode;
}

void insertAtHead(Node** head, int data) {
    Node* newNode = createNode(data);
    newNode->next = *head; // Point newNode's next to the current head
    if (*head != NULL) {
        (*head)->prev = newNode; // Update current head's prev to newNode
    }
    *head = newNode; // Update the head to point to the newNode
}
```

```
int main() {
    Node* head = NULL; // Start with an empty list

    // Example usage
    insertAtHead(&head, 10); // List: 10
    insertAtHead(&head, 20); // List: 20 -> 10
    insertAtHead(&head, 30); // " 30 -> 20 -> 10

    printf("Doubly Linked List: ");
    printList(head);

    return 0;
}
```

BCIT Coding For All

General Tips: Practice

- Practice relevant leetcode everyday in C
- Get good with pointers, linked lists, maybe arrays
- Practice thinking through the logic of solving problems
- Get reasonable sleep, it will help your logical thinking

BCIT Coding For All

General Tips: Midterm

- Easy to pass, hard to ace
- Think a little before writing
- Start with the easiest question
- There will be some gimmes, don't mess them up!
- You can get marks by just explaining your logic in comments/pseudocode on written questions
- There will be restrictions for the questions e.g "max 5 lines", "in O(log(n)) time", etc...

BCIT Coding For All

Before we Begin Practice Problems

It’s important to know the things we’ve reviewed to this point, but the most important thing to prepare you for this midterm is practicing ~~linked list~~ arrays, structs and pointer problems using leetcode or other resources.

BCIT Coding For All

Practice Problems: Pointers

Write a function that swaps the contents of pointers. Make sure to write any missing information in the code snippet below. Your solution in swapPointers should not be more than 5 lines of code.

```
void swapPointers( ) {
//max 5 lines of code
/*
params empty to start, you decide what to put there, but it
must compile and swap the pointers
*/



}
```

```
int main(){
int firstNumber = 1;
int secondNumber = 2;

int *firstPointer =
int *secondPointer =

swapPointers();

return 0;
}
```

Solution: Pointers

Write a function that swaps the contents of pointers. Make sure to write any missing information in the code snippet below. Your solution in swapPointers should not be more than 5 lines of code.

```
void swapPointers(int **f, int **s) {
    int *temp = *f;
    *f = *s;
    *s = temp;
}
```

```
int main() {
    int firstNumber = 1;
    int secondNumber = 2;

    int *firstPointer = &firstNumber;
    int *secondPointer = &secondNumber;

    swapPointers(&firstPointer, &secondPointer);

    return 0;
}
```

Practice Problems: Pointers II

What is the output of the following code?

```
#include<stdio.h>
int main() {
    int x = 7;
    int *p = &x;
    int **pp = &p;

    printf("%d\n", *p);
    printf("%d\n", **pp);

    *p = 15;
    printf("%d\n", *p);
    printf("%d\n", **pp);

    return 0;
}
```

Practice Problems: Pointers II Solution

What is the output of the following code?

```
#include<stdio.h>
int main() {
int x = 7; // x initialized to 7
int *p = &x; // p points to x

int **pp = &p; // pp points to p
printf("%d\n", *p); // prints value of x (7)
printf("%d\n", **pp); // prints value of x (7) via pp

*p = 15; // updates x to 15 via p
printf("%d\n", *p); // prints updated value of x (15)
printf("%d\n", **pp); // prints updated value of x (15) via pp
return 0;
}
```

```
7
7
15
15
```

Practice Problems: Pointers III

What best describes the following: **int * (* (* * x [ ] [ ]) ( )) [8];**

A) x is array of array of 8 pointers to functions returning pointer to array of pointer to a pointer to int

B) x is a function that takes as input an array of 8 pointers to pointer to an array and returns a pointer to array of pointer to int

C) x is an array of pointers to pointer to function returning 8 pointers to array of pointer to int

D) x is an array of 8 pointers to functions, where each function returns a pointer to a function returning a pointer to a function returning a pointer to an integer

Practice Problems: Pointers III

What best describes the following: **`int * ( * ( * * x [ ] [ ] ) ( ) ) [8];`**

D) x is an array of 8 pointers to functions, where each function returns a pointer to a function returning a pointer to a function returning a pointer to an integer

`x [ ] [ ]`: x is a **two-dimensional array**. So, x is an array of arrays of **something**.

`* * x [ ] [ ]`: The `* *` before x indicates that each element of this 2D array is a **pointer to a pointer** to something. So x is a 2D array of **pointers to pointers**.

`( * * x [ ] [ ] ) ( )`: The parentheses and empty `( )` indicate that this pointer-to-pointer (each element in x) is actually a **function pointer**. So, each element in x is a pointer to a **function** that takes **no arguments**.

`* ( * * x [ ] [ ] ) ( )`: This outer `*` indicates that the function (pointed to by the elements of x) **returns a pointer**.

`* ( * ( * * x [ ] [ ] ) ( ) ) [8]`;
Now, this part indicates that the function returns a **pointer to an array of 8 pointers**.

`int *`:
Finally, the `int *` at the beginning tells us that this pointer (returned by the function) points to an **integer**.

Practice Problems: Arrays

What is the output of the following code?

```
#include <stdio.h>
void transformArrays(int A[], int B[], int n) {
        for (int i = 0; i < n; i++) {
              if (i % 2 == 0) {
                    A[i] = A[i] + B[n - 1 - i];
              } else {
                    B[i] = A[i] - B[i];
              }
          }
}
void printArray(int arr[], int n) {
      for (int i = 0; i < n; i++) {
            printf("%d ", arr[i]);
      }
      printf("\n");
}
```

```
int main() {
      int A[] = {5, 10, 15, 20, 25};
      int B[] = {1, 2, 3, 4, 5};
      int n = 5;
      transformArrays(A, B, n);

      printf("Array A: ");
      printArray(A, n);

      printf("Array B: ");
      printArray(B, n);
      return 0;
}
```

Practice Problems: Arrays

What is the output?

```
Array A: 10 10 18 20 26
Array B: 1 8 3 16 5
```

Problem: General Logic
What is the output of the following code?

```
#include <stdio.h>

int main() {
    int new_board[4][4];
    int x = 0, y = 0;
    int i, j;

    for (i = 0; i < 4; i++) {
        for (j = 0; j < 4; j++) {
            new_board[i][j] = i * 4 + j + 5;
        }
    }
//continued
```

```
for (i = 0; i < 10; i++) {
        if (i % 4 == 0) {
            x = (x + 2) % 4;
        } else if (i % 4 == 1) {
            y = (y + 3) % 4;
        } else if (i % 4 == 2) {
            new_board[x][y] += new_board[3 - x][3 - y];
        } else {
            new_board[x][y] -= 1; // Subtract 1 from the current value
        }
        printf("%d ", new_board[x][y]);
    }
    printf("\n");

    return 0;
}
```

Problem: General Logic
What is the output of the following code?

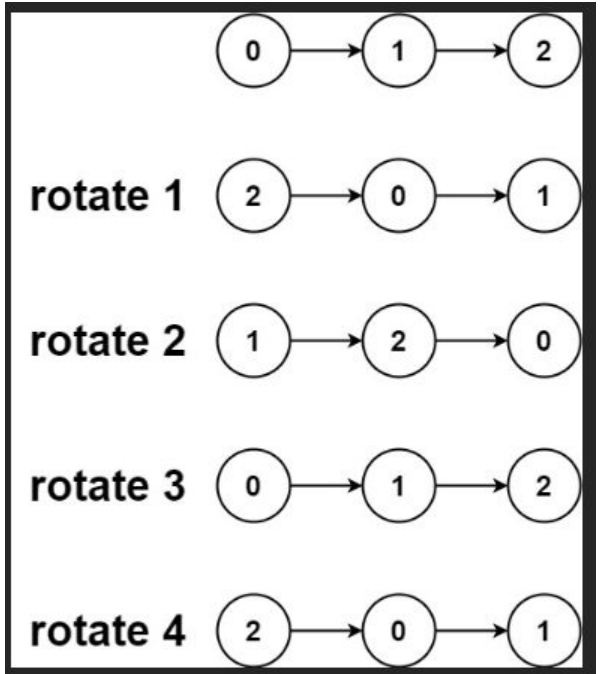
The output is as follows:

13 16 25 24 8 7 25 24 15 14

Practice Problems: Linked List

Given the head of a linked list, rotate the list to the right by k places.

```
 Typedef struct ListNode {
     int val;
     struct ListNode *next;
}Node;

struct ListNode* rotateRight(struct ListNode*
head, int k) {

}
```



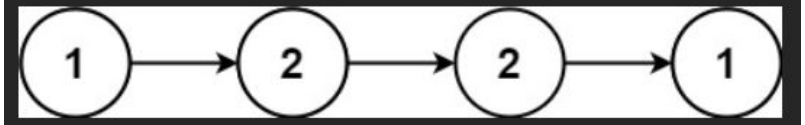
Input: head = [0,1,2], k = 4
Output: [2,0,1]

Practice Problems: Linked List (2)

Given the head of a singly linked list, return true *if it is a Palindrome or* false *otherwise*.

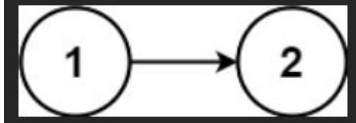
```
 Typedef struct ListNode {
     int val;
     struct ListNode *next;
}Node;

bool isPalindrome(struct ListNode* head) {

}
```

Example 1:



Input: head = [1,2,2,1]
Output: true

Example 2:



Input: head = [1,2]
Output: false

Questions?

BCIT Coding For All

GOOD LUCK TO YOU ALL!

Thanks for coming!

BCIT Coding For All